

# Pyramid

A task and project workspace built with Next.js and NestJS, backed by MongoDB Atlas. This document walks the application screen by screen, traces how a change reaches the database, and records what was verified — and what was not.



Janmejy Mahato

21 August 2026

Next.js 16 • NestJS 11 • MongoDB

01

## What this application does

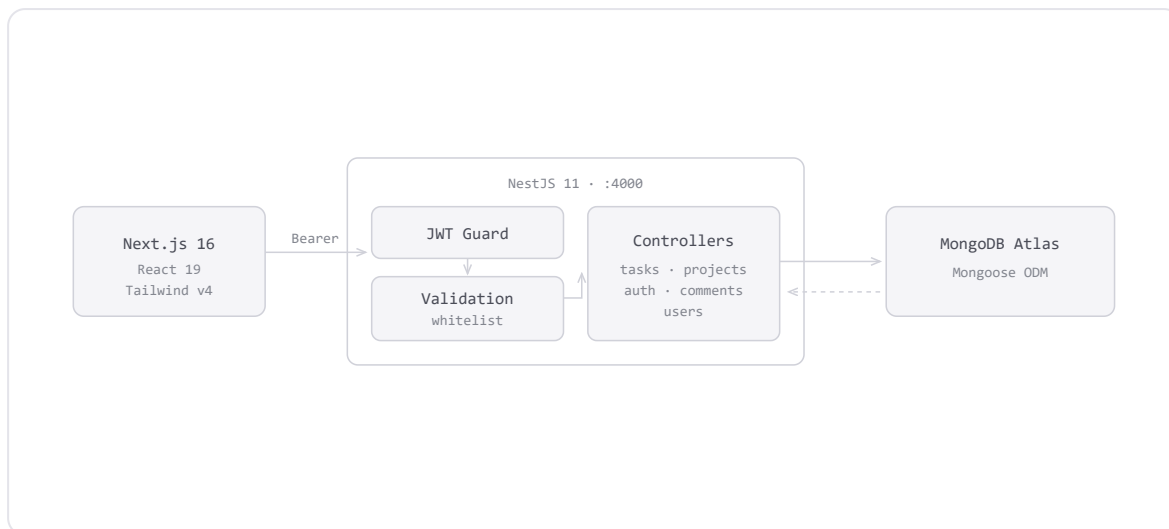
Pyramid is a workspace for tracking tasks and projects. A user signs in, sees their tasks grouped by status, and moves work through four stages. The same data is presented two ways — grouped tables and Kanban columns — and every task opens into a detail page carrying description, labels, subtasks, comments and an activity feed.

The governing design goal is that **nothing is a local-only illusion**. Every visible control writes through to the database. Change a priority in a dropdown, reload the page, and the change is still there. Section 11 records the test run that proves this for all 21 interactive paths.

02

# Architecture

Two packages — a Next.js frontend and a NestJS API — talking REST, with MongoDB Atlas for storage. Requests cross three gates before touching data: token verification, DTO validation, then the service layer.



## Decisions worth naming

**One grouped endpoint feeds both views.** `GET /api/tasks/grouped` returns tasks already bucketed by status. List and board render the same payload, so the two can never disagree, and switching between them costs no request.

**Mutations re-fetch instead of patching local state.** Every write calls the API then reloads the grouped payload — trading a little latency for the guarantee that what you see is what the database holds.

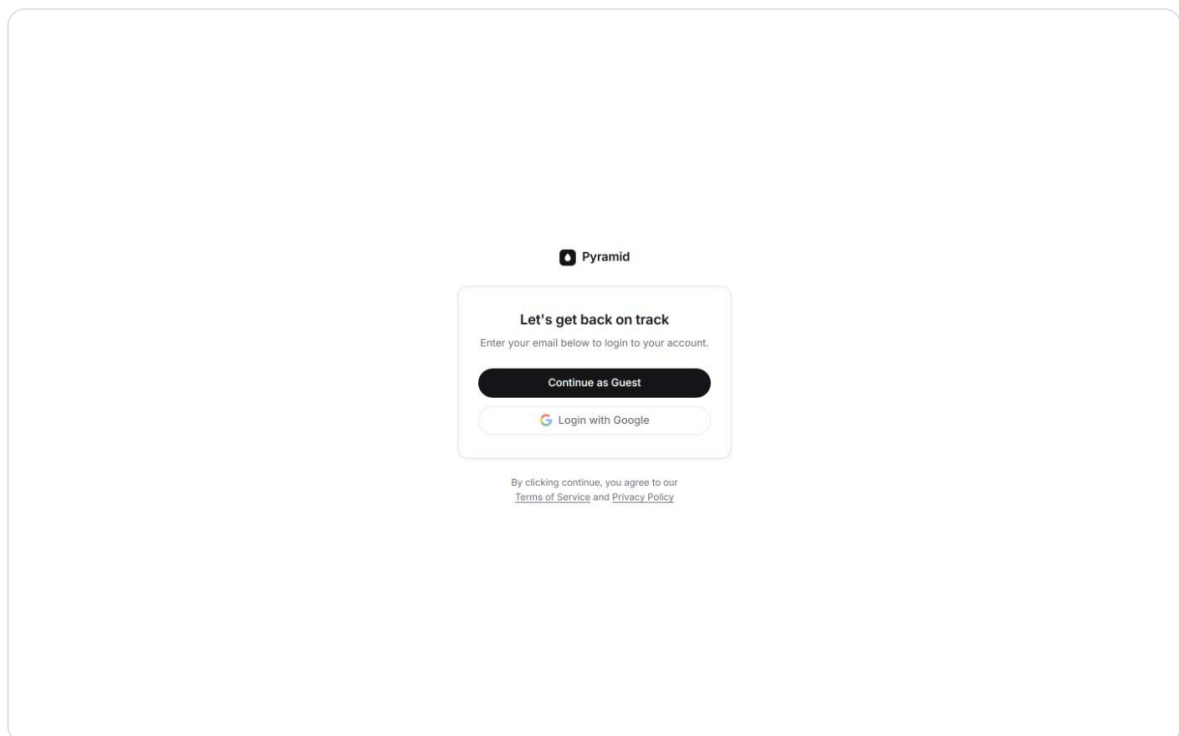
**The vocabulary is defined once per side.** Four statuses, five priorities, declared in `constants.ts` and enforced at the validation boundary. Note the deliberate casing split: statuses are Title Case ( `"To Do"` ), priorities lowercase ( `"high"` ).

**Theming is two independent axes.** `data-theme` and `data-accent` are separate attributes driving CSS custom properties. No component hardcodes a colour.

# Authentication

**Continue as Guest** calls `POST /api/auth/guest`, which creates a throwaway user document and returns a signed JWT. The token lands in `localStorage`; on each subsequent load `AuthProvider` resolves it via `GET /auth/me`. An expired or revoked token is discarded and the app falls back to anonymous rather than erroring.

Each guest login creates a *distinct* user, so two people testing never share a board.



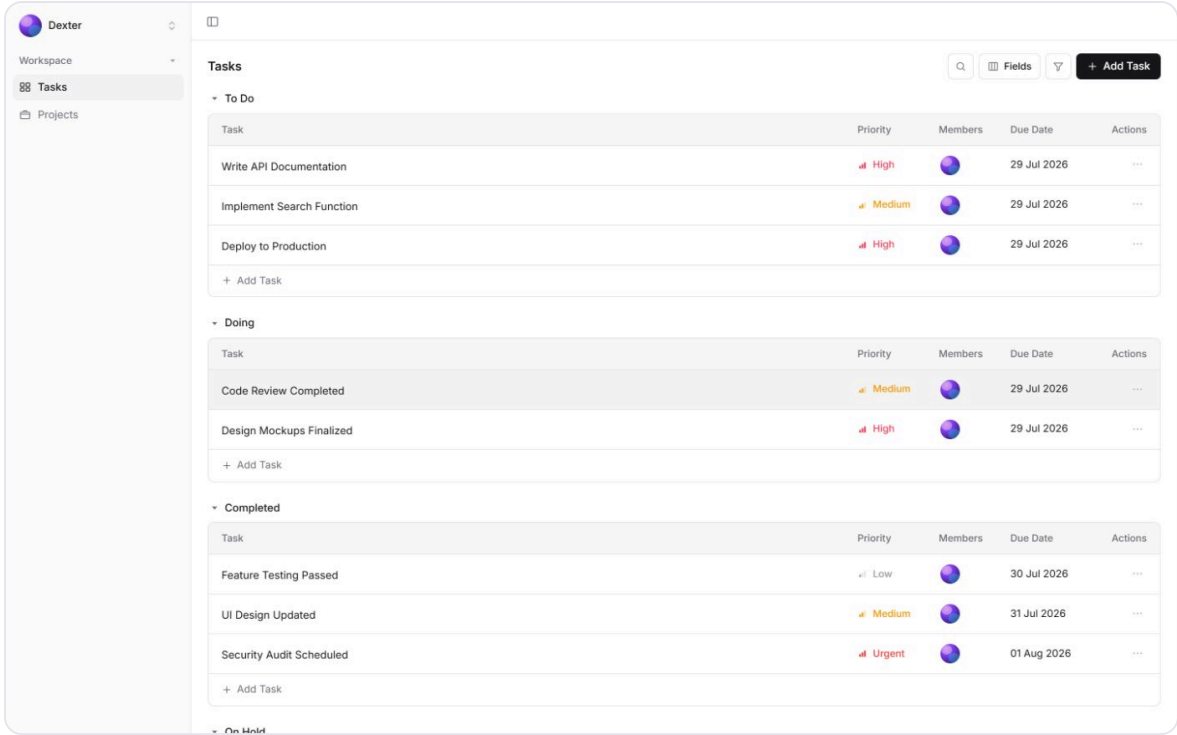
The entry screen. Google login renders to match the design but is not wired – see section 13.

---

04

## The task list

The default landing view. Tasks are grouped into one table per status, each with its own header and its own **Add Task** affordance. Priority renders as a coloured glyph plus label, the colour carrying the semantics.



Four status groups, each an independent table. Optional columns are controlled from the Fields menu.

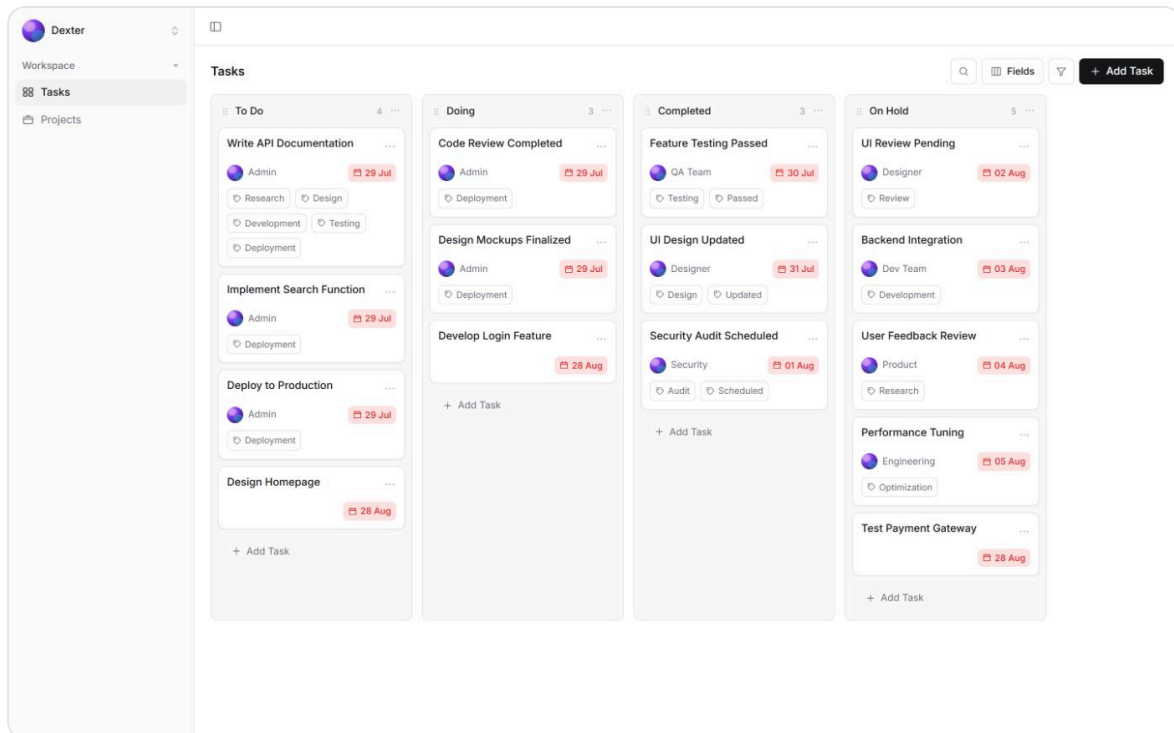
**Inline creation.** *Add Task* opens a text field in place; Enter creates the task in that group's status with a due date defaulted one week out, so a new row is never missing a date.

**Row actions.** The ... menu exposes priority, status and delete. Changes write immediately and relocate the row if its group changed.

05

## Board view

The same four statuses as columns, each showing a count. Cards surface more than list rows do: assignee, a due-date chip, and labels as individual chips. The due-date chip carries a red-tinted background — this design treats due dates as attention-carrying, not neutral metadata.

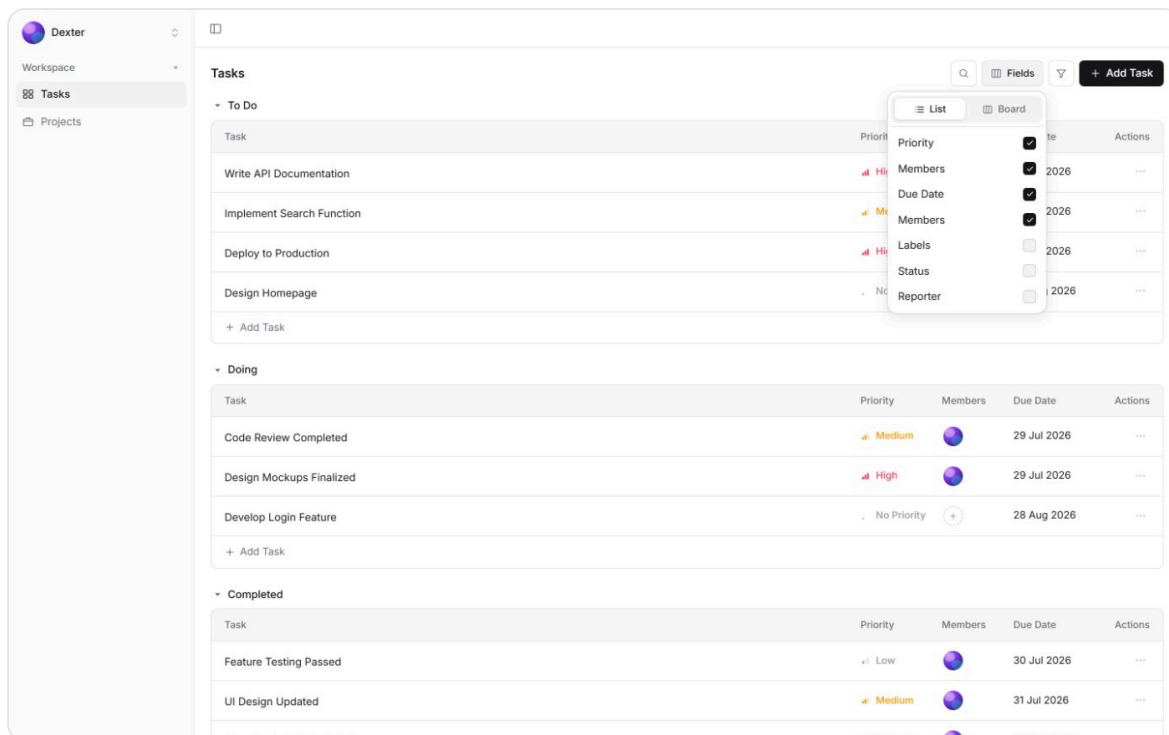


Board view. Same payload as the list, no refetch on switch.

06

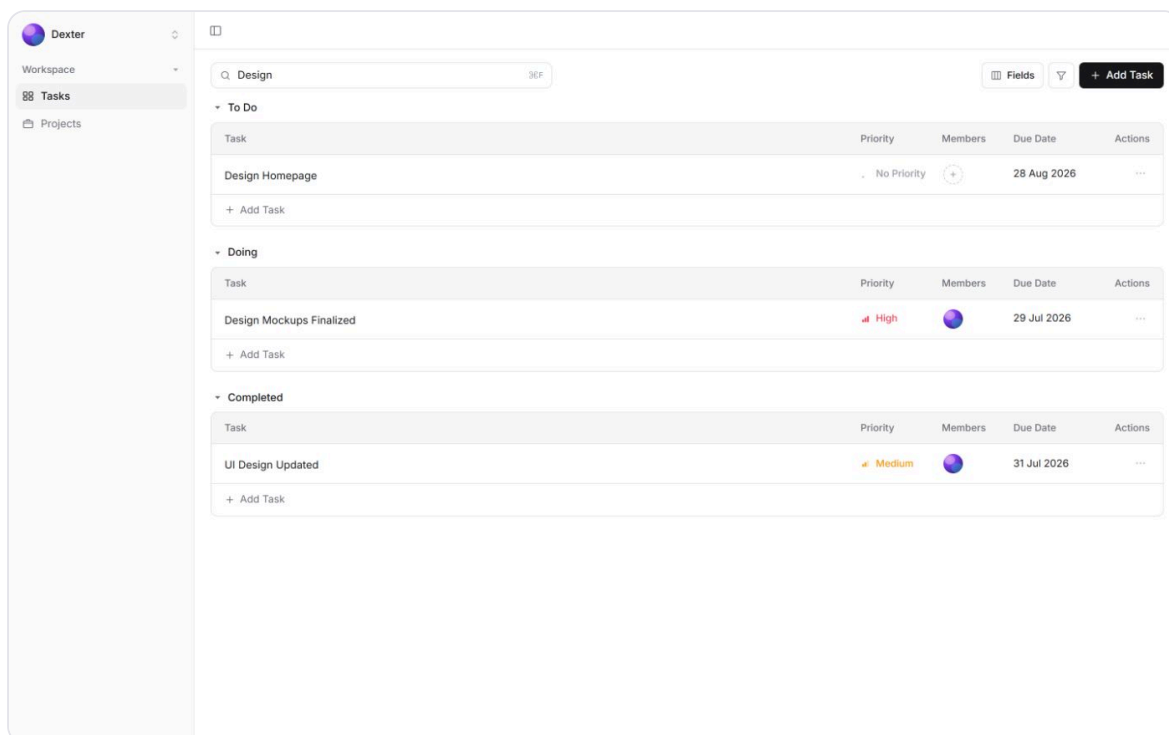
## Search and filtering

**Fields** carries both the List/Board switch and per-column visibility.



The Fields menu. Note the two rows both labelled Members – see improvement 4.

**Search** opens from the magnifier or `⌘F` / `Ctrl+F`, intercepted so the app's search opens instead of the browser's. Input is debounced, and filtering runs client-side against the already-grouped payload, so results are instant.

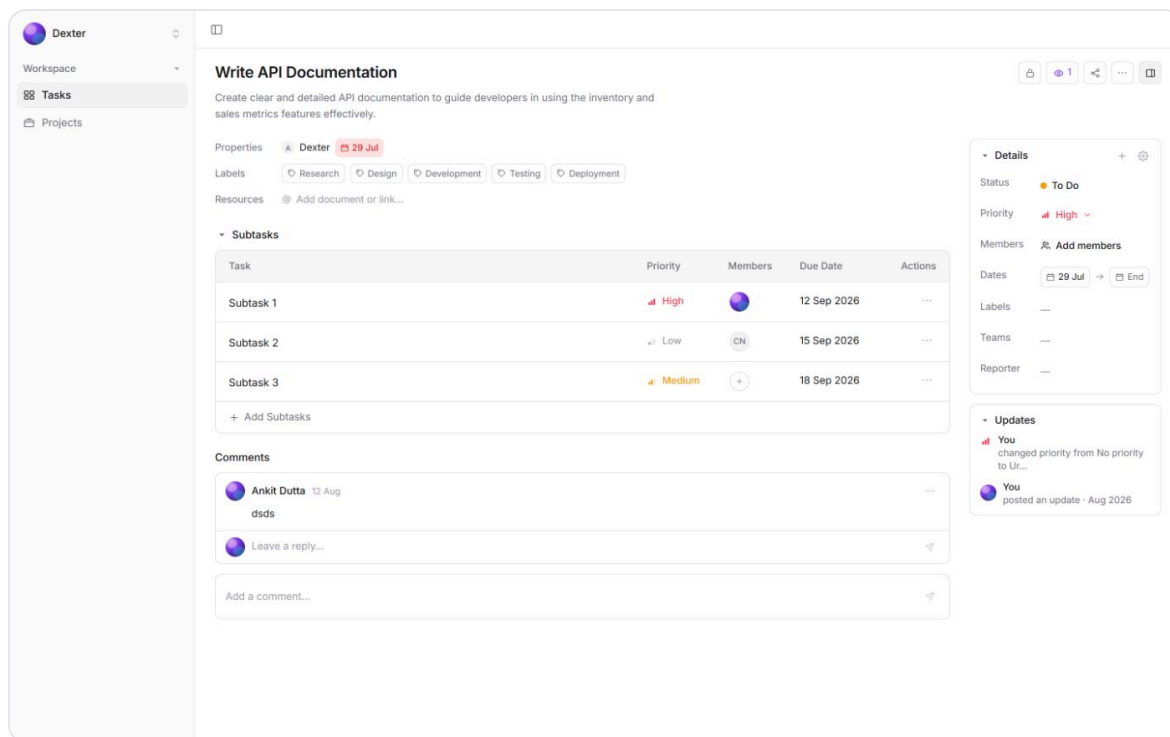


Search active – the page title collapses into the input. Empty groups hide while filtering.

07

## Task detail

A two-column layout: content left, **Details** panel right. Title and description are click-to-edit, saving on blur — there is no save button because the design shows none.



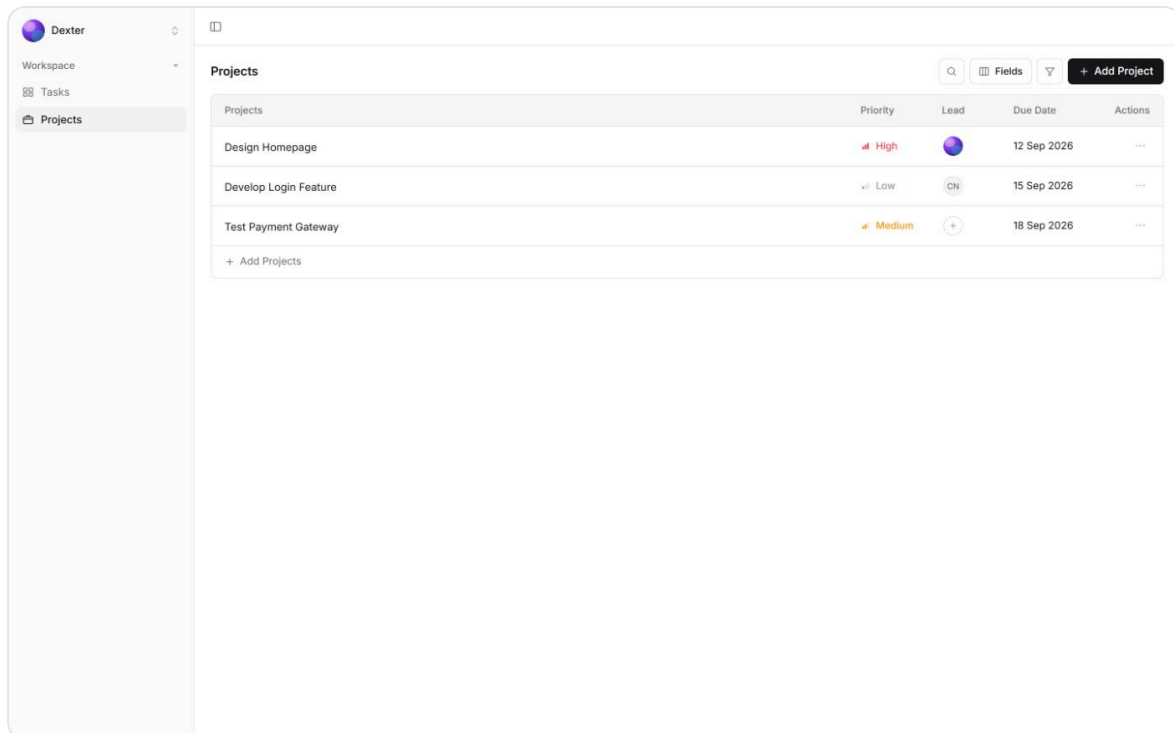
Task detail, captured after the improvement-1 fix – the subtask table and comment thread now carry distinct headings.

**Subtasks** reuse the same table component as the main list, so editing behaves identically at both levels. **Comments** support threaded replies; the composer retains its text if a post fails. **Updates** shows a server-recorded activity feed.

08

## Projects

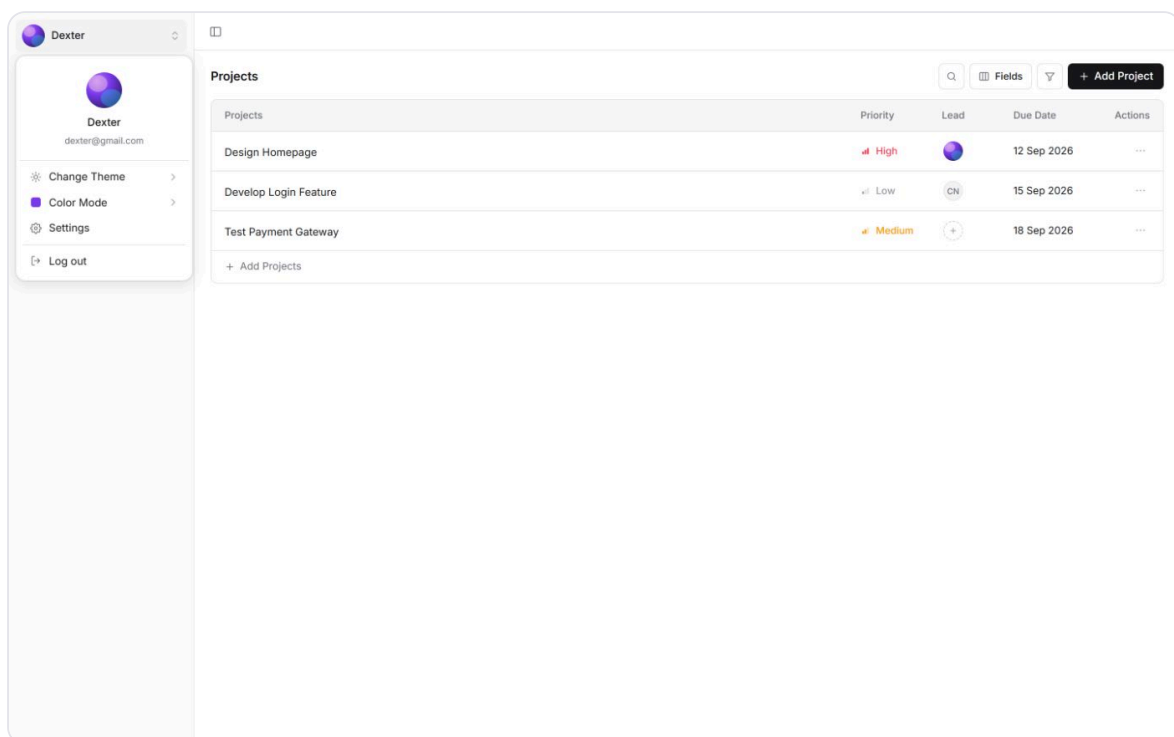
The same table shell as tasks, with Priority, Lead and Due Date. Each project opens to a detail page listing that project's tasks — the task list scoped by `projectId`.



Projects. The Lead cell shows all three member states: avatar, initials fallback (CN), and unassigned.

09

## Settings and theming



The user menu – theme, colour, settings, logout.



Profile fields save on blur through `PATCH /api/users/me`, verified to survive a reload. Theme switches light/dark; Color picks among six accents.

← Back to app

Q Search

Profile

Theme

Color

### Profile

Profile picture

Email

Full name Dexter

Title  
Your job title or role

Username  
One word, like a nickname or first name

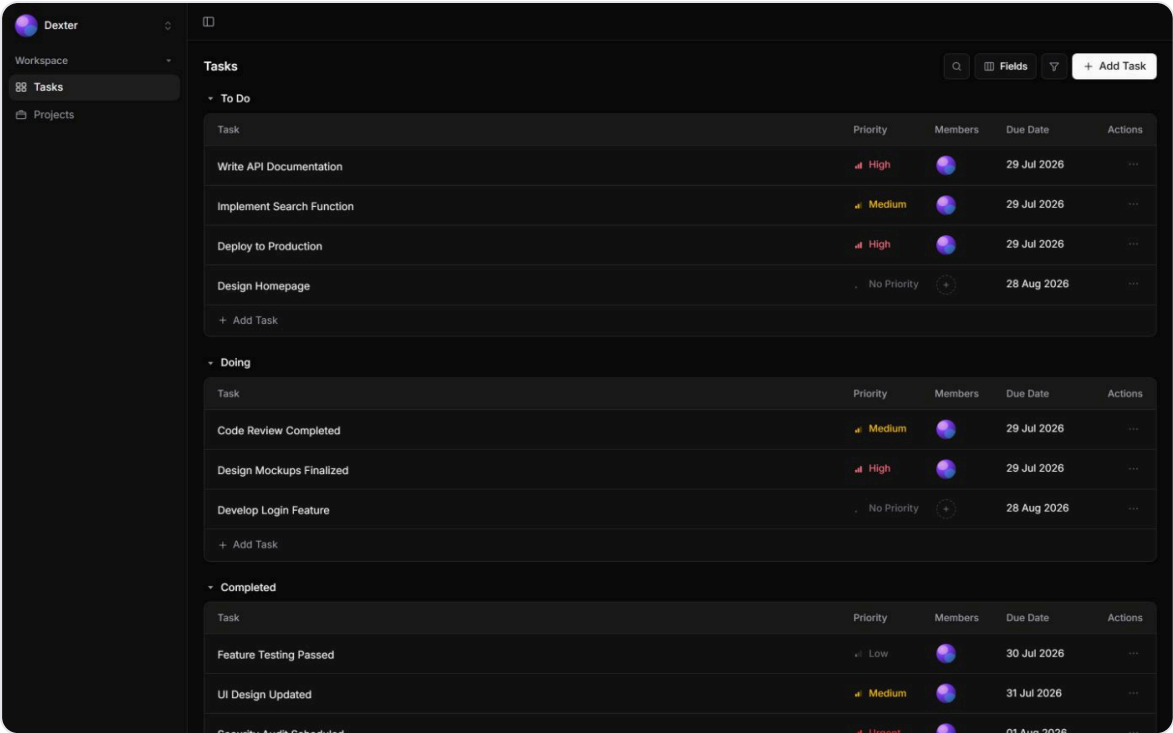
### Workspace access

Remove yourself from the workspace

Leave Workspace

Settings. Editable fields with no save button, matching the design.

Dark mode is a genuine re-themed palette rather than an inversion — surfaces, borders, table headers and priority colours all carry separate dark values, with priority tones brightened for contrast.



Dark mode. Priority hues shift lighter to hold contrast against the dark ground.

10

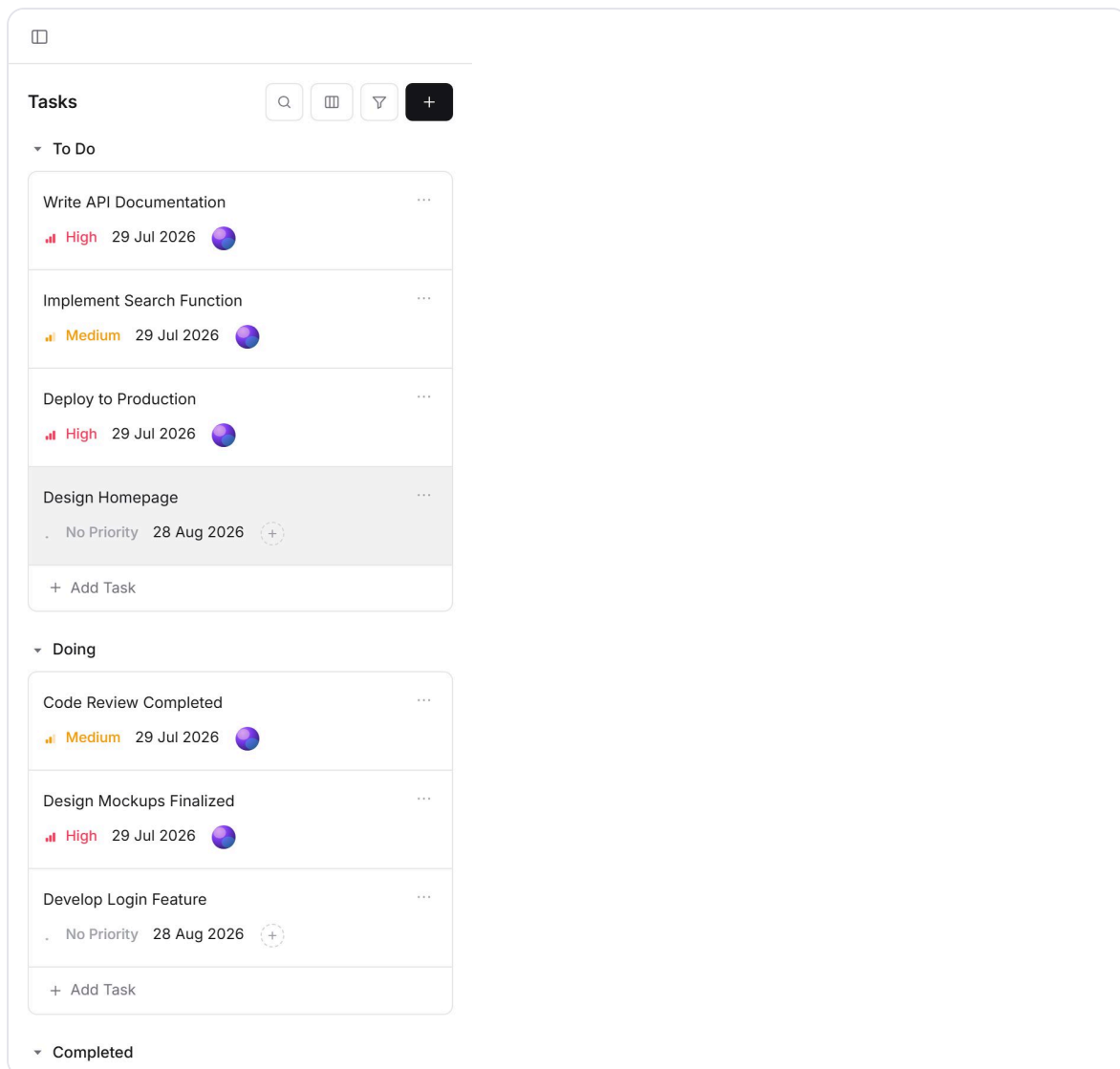
## Responsive behaviour

Verified at five widths by comparing `scrollWidth` against `clientWidth`. No horizontal overflow at any breakpoint.

| WIDTH | LAYOUT BEHAVIOUR                      | OVERFLOW |
|-------|---------------------------------------|----------|
| 1440  | Full two-pane, sidebar 228px          | None     |
| 1280  | Same, sidebar 210px                   | None     |
| 1024  | Same, toolbar labels intact           | None     |
| 768   | Sidebar becomes overlay drawer        | None     |
| 390   | Tables restructure into stacked cards | None     |

The mobile treatment is the notable one. Rather than horizontally scrolling a five-column table, each row becomes a card — title on one line, priority, date and

assignee on a second. Toolbar buttons drop their labels; the sidebar becomes a drawer with scrim, scroll lock and Escape-to-close.

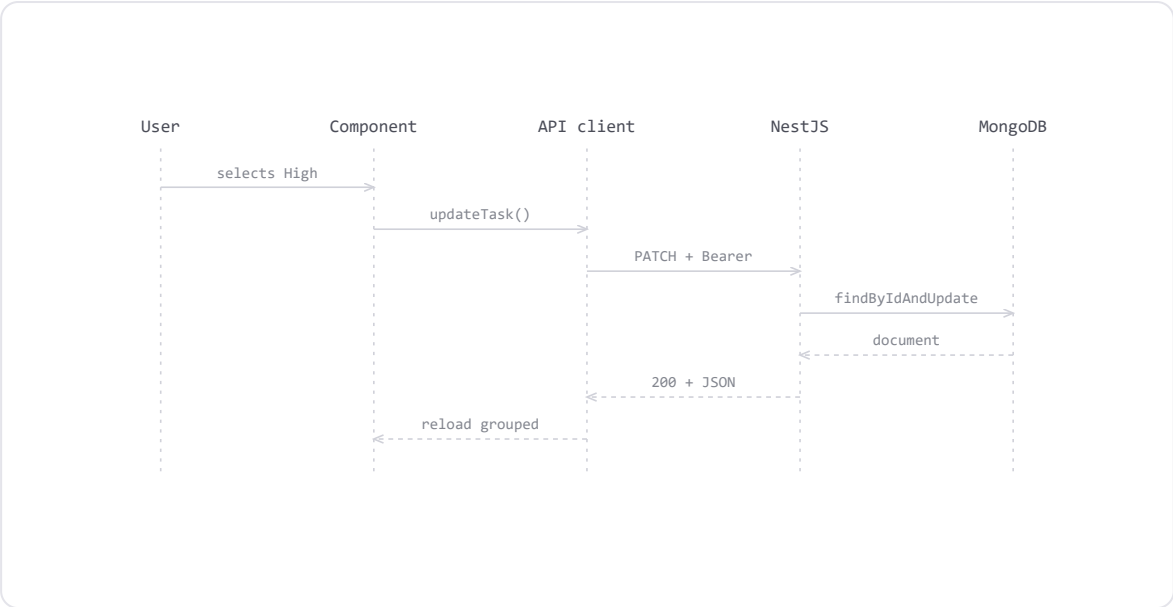


390px. The table restructures rather than scrolling sideways.

11

## How a change persists

Changing a priority from a row menu, end to end:



The re-fetch at the end is what keeps list and board consistent without duplicating optimistic-update logic per view.

The validation boundary

A global `ValidationPipe` runs with `whitelist` and `forbidNonWhitelisted`, so an unknown field is a 400 rather than a value that silently does nothing. Errors flow through `AllExceptionsFilter` into a consistent shape, and the client's `ApiError` carries the status code so callers can tell 401 from 400.

12

Verification

Every figure below was produced by running the checks, not estimated.

|                                                           |                                                     |                                                      |
|-----------------------------------------------------------|-----------------------------------------------------|------------------------------------------------------|
| <div>21/21</div> <div>Interaction assertions passed</div> | <div>0</div> <div>Console errors, all screens</div> | <div>0</div> <div>Lint problems, both packages</div> |
| <div>5/5</div> <div>Breakpoints without overflow</div>    |                                                     |                                                      |

| CHECK                                         | RESULT     |
|-----------------------------------------------|------------|
| Frontend TypeScript <code>tsc --noEmit</code> | Clean      |
| Backend TypeScript                            | Clean      |
| Frontend production build, 7 routes           | Success    |
| Backend build <code>nest build</code>         | Success    |
| ESLint, frontend and backend                  | 0 problems |
| <code>GET /api/health</code>                  | 200        |
| <code>GET /api/tasks</code> unauthenticated   | 401        |
| <code>POST /api/auth/guest</code>             | 201        |
| CRUD round-trip, create to confirm-404        | Passed     |

## Interaction suite

`frontend/test/ui-check.mjs` drives every control in real Chromium and asserts each change persisted through the API.

```

=== login ===
    PASS guest login navigates to /tasks
    PASS seeded data renders from MongoDB
=== create task ===
    PASS Add Task opens an inline field
    PASS new task appears
=== row actions ===
    PASS row actions menu opens
    PASS priority change persists
    PASS status move relocates the task
=== board view ===
    PASS board renders columns
    PASS task visible on board
=== task detail ===
    PASS navigates to task detail
    PASS title edit persists
    PASS description edit persists
=== details panel ===
    PASS status dropdown opens
    PASS members picker opens
=== subtasks + comments ===
    PASS subtask created
    PASS comment posted
=== projects ===
    PASS project created
    PASS project deleted
=== settings + theme ===
    PASS profile name persists after reload
    PASS theme switch applies
=== cleanup ===
    PASS task deleted

passed: 21  failed: 0
console errors: none

```

13

## Improvements identified

Ordered by impact. The first is a defect found while producing this document, since fixed; the rest are opportunities.

## DEFECT FOUND DURING WALKTHROUGH — FIXED

**The comments section was labelled “Subtasks”**

On task detail, the comments heading repeated the label of the subtask table directly above it — two adjacent sections carrying the same heading. The surrounding code comment already read `{/* Comment thread */}`, confirming the label was wrong rather than intentional.

`task-detail-view.tsx:368` — **Fixed:** the heading now reads “Comments”.

Verified in the running application — the page renders exactly one

`Subtasks` heading and one `Comments` heading — with the interaction suite still passing 21/21. The section 7 screenshot shows the corrected state.

**02 No drag-and-drop on the board**

UX GAP

Column headers already render a grip handle implying it, but status changes require the ... menu. **Suggested:** pointer drag with a keyboard-accessible fallback. The largest gap relative to what a Kanban board implies.

**03 Most filter categories are inert**

UX GAP

The Filter menu lists six categories but only Priority has a working flyout; the rest open empty submenus. **Suggested:** implement or hide them — a menu item that does nothing is worse than an absent one.

**04 The Fields menu lists “Members” twice**

CLARITY

Two entries share a label while toggling different columns (members and assignees). This mirrors the design, but in a live product it confuses. **Suggested:** relabel to “Members” and “Assignees”.

**05 No optimistic UI on mutations**

POLISH

Every change round-trips before the UI updates — imperceptible locally, sluggish on a slow connection. **Suggested:** apply locally and reconcile on response. Worth doing only with real rollback; the current approach is correct, just not fastest.

---

## 06 Mutation failures are silent UX GAP

The change handlers `await` without catching, so a failed request leaves the UI unchanged and unexplained. **Suggested:** a failure toast. The comment composer already models the right behaviour by keeping its text.

---

## 07 No loading skeletons POLISH

Views render loading states but not shaped placeholders, so there is an empty flash before data arrives — most visible on the ~20s Atlas cold start. **Suggested:** skeleton rows at final row height, which also prevents layout shift.

---

## 08 Delete has no confirmation or undo UX GAP

Deleting from the ... menu is immediate and irreversible. **Suggested:** an undo toast — less interruptive than a dialog and it recovers the more common regret.

---

## 09 Production hardening on the API PRE-DEPLOY

Not needed for an assessment, noted for completeness. **No rate limiting** — `POST /auth/guest` writes a user document per call and is unauthenticated, so it is trivially spamnable; `@nestjs/throttler` fixes it. **No security headers** — `helmet` is one line. **Pagination exists but is unused** — `PaginationDto` supports `skip/take` capped at 100, but the UI never paginates, so a board past 100 tasks would silently truncate.

---

## 10 Accessibility is strong with two gaps POLISH

ARIA is used conscientiously — 30 `aria-label`, 11 `aria-expanded`, 8 `aria-haspopup`, 6 `aria-checked`, plus `aria-current` and `aria-modal`; menus close on Escape and the drawer locks scroll. **Remaining:** several custom controls lack a visible focus ring, and the mobile drawer has no focus trap, so tabbing escapes to content behind it.

---



# Known limitations

**Design comparison against the Figma source was not performed.** The Figma file for this assessment could not be opened with the access available during development. This document therefore describes the implementation as built and verified; it makes no claim of pixel-level equivalence to the Figma design, and no measurement audit was carried out.

**Google login is not implemented.** The button renders to match the design; authentication is guest-only.

**Some detail-header controls are presentational.** Lock, watcher count and share render per the design but are not wired to behaviour.

**Atlas cold start takes roughly 20 seconds.** Mongoose logs one retry before connecting. Harmless locally, but a platform with a short startup healthcheck could fail on it.

**No unit test suite.** `jest` finds no `.spec.ts` files; coverage comes from the end-to-end harnesses. `npm test` in the backend exits non-zero on an empty suite.

---

15

## Running locally

```
# Backend — needs backend/.env with DATABASE_URL and JWT_SECRET
cd backend
npm install
npm run build
npm run start:prod           # http://localhost:4000/api

# Frontend — needs frontend/.env.local with NEXT_PUBLIC_API_URL
cd frontend
npm install
npm run build
npm start                   # http://localhost:3000
```

Both `.env.example` files are committed as templates. To run the interaction suite with both servers up:

```
cd frontend
npm i -D playwright && npx playwright install chromium
APP_URL=http://localhost:3000 node test/ui-check.mjs
```

---

Pyramid — walkthrough documentation. Screenshots captured from the running application at 1440×900, device scale 2. All verification figures reflect actual command output.